

Algorithms and Data Structures

Problem Sheet 10

Hermann Schichl

90. Write pseudocode for the procedure `CONSTRUCT-OPTIMAL-BST( $root, n$ )` which, given the table `root[1 :  $n$ , 1 :  $n$ ]` produced by the function `OPTIMAL-BST( $p, q, n$ )`, constructs the optimal binary search tree.
91. Determine the cost and structure of an optimal binary search tree for the keys k_i and failed searches d_i :

i	0	1	2	3	4	5	6	7
p_i	0.04	0.06	0.08	0.02	0.10	0.12	0.14	
q_i	0.06	0.06	0.06	0.06	0.05	0.05	0.05	0.05

92. In the Euclidean traveling-salesperson problem, you are given a set of n points in the plane, and your goal is to find the shortest closed tour that connects all n points. The general problem is NP-hard, as discussed in the lecture course, and its solution is therefore believed to require more than polynomial time. J. L. Bentley has suggested simplifying the problem by considering only *bitonic tours*, that is, tours that start at the leftmost point, go strictly rightward to the right-most point, and then go strictly leftward back to the starting point, see also *Introduction to Algorithms, Fourth Edition, Figure 14.11 on page 408*. In this case, a polynomial-time algorithm is possible. Describe an $O(n^2)$ -time algorithm for determining an optimal bitonic tour. You may assume that no two points have the same x -coordinate and that all operations on real numbers take unit time. (Hint: Scan left to right, maintaining optimal possibilities for the two parts of the tour.)
93. Professor Blutarsky is consulting for the president of a corporation that is planning a company party. The company has a hierarchical structure, that is, the supervisor relation forms a tree rooted at the president. The human resources department has ranked each employee with a conviviality rating, which is a real number. In order to make the party fun for all attendees, the president does not want both an employee and his or her immediate supervisor to attend. Professor Blutarsky is given the tree that describes the structure of the corporation, using the left-child, right-sibling representation described in Section 10.3. Each node of the tree holds, in addition to the pointers, the name of an employee and that employee's conviviality ranking. Describe an algorithm to make up a guest list that maximizes the sum of the conviviality ratings of the guests. Analyze the running time of your algorithm.

94. Consider the activity selection (room scheduling) problem presented in the lecture course. Suppose that instead of always selecting the first activity to finish, you instead select the last activity to start that is compatible with all previously selected activities. Describe how this approach is a greedy algorithm, and prove that it yields an optimal solution.
95. Consider again the activity selection (room scheduling) problem. Not just any greedy approach to the activity-selection problem produces a maximum-size set of mutually compatible activities. Give an example to show that the approach of selecting the activity of least duration from among those that are compatible with previously selected activities does not work. Do the same for the approaches of always selecting the compatible activity that overlaps the fewest other remaining activities and always selecting the compatible remaining activity with the earliest start time.
96. You are given a set of activities to schedule among a large number of lecture halls, where any activity can take place in any lecture hall. You wish to schedule all the activities using as few lecture halls as possible. Give an efficient greedy algorithm to determine which activity should use which lecture hall. (This problem is also known as the interval-graph coloring problem. It is modeled by an interval graph whose vertices are the given activities and whose edges connect incompatible activities. The smallest number of colors required to color every vertex so that no two adjacent vertices have the same color corresponds to finding the fewest lecture halls needed to schedule all of the given activities.)
97. Consider a modification to the activity-selection problem in which each activity a_i has, in addition to a start and finish time, a value v_i . The objective is no longer to maximize the number of activities scheduled, but instead to maximize the total value of the activities scheduled. That is, the goal is to choose a set A of compatible activities such that $\sum_{a_k \in A} v_k$ is maximized. Give a polynomial-time algorithm for this problem.
98. Prove that the fractional knapsack problem has the greedy-choice property. Produce an algorithm that solves the problem in $O(n)$ time.
99. Give a dynamic-programming solution of the 0 – 1 knapsack problem that runs in $O(nW)$ time, where n is the number of items and W is the maximum weight of items that the thief can put in the knapsack.